INTERVIEW DEFENSE DOCUMENT

DocIngestQA

v0.2.0

Pre-indexing QA auditor for RAG document ingestion pipelines.
11 deterministic checks for missing pages, OCR noise, duplicates,
encoding corruption, and poor split boundaries.

Metric	Value
Checks (v0.1)	8 deterministic checks
Checks (v0.2)	11 deterministic checks (+3 new)
Demo corpus	64 chunks, 10 documents
Demo overall status	FAIL (10 missing pages detected)
Demo check results	4 PASS, 6 WARN, 1 FAIL
Output formats	JSON, Markdown, HTML
CLI	python -m docingestqa chunks.jsonl
PyPI package	pip install docingestqa
Python support	3.10, 3.11, 3.12
Dependencies	jinja2 only (zero heavy deps)

Sidharth Kriplani | May 2026

1. Executive Summary

DocIngestQA is a pip-installable Python library that audits RAG document ingestion pipelines before chunks reach the vector database. It answers one question: did the ingestion pipeline silently lose pages, create empty chunks, duplicate content, corrupt text with encoding errors, or produce poorly-split fragments?

The library takes a JSONL file of chunks and an optional source manifest, runs 11 deterministic checks, and produces structured JSON/Markdown/HTML output with chunk-level evidence. Every issue is locatable: you get the source file, page number, and text preview for each flagged chunk.

v0.2.0 adds three new checks — chunk overlap detection, encoding health (mojibake/null bytes), and split quality — plus a CLI and Python 3.10 compatibility.

1.1 Why this exists

RAG pipelines fail silently at ingestion. A missing page means a question about that page will hallucinate. A duplicate chunk skews retrieval rankings. Mojibake encoding corruption produces wrong answers even when the right chunk is retrieved. Mid-sentence splits cause generation models to produce incomplete answers.

These failures are cheap to catch before indexing and expensive to debug after deployment. DocIngestQA is the pre-indexing gate that most RAG teams build ad-hoc and never package.

1.2 Design principles

Deterministic. No ML models, no embeddings, no probabilistic outputs. Every check produces a reproducible result on the same input.

Dependency-light. The only runtime dependency is jinja2 (for HTML reports). No numpy, no pandas, no transformers. Works in any Python environment.

Chunk-level evidence. Issues are reported with source, page, and text preview — not just aggregate counts. You can trace every issue back to a specific chunk.

Honest interpretation note. Every output includes a mandatory, unsuppressable note: DocIngestQA reports ingestion quality signals, not retrieval relevance or answer faithfulness. This boundary cannot be removed.

2. Architecture

DocIngestQA is structured as a single-responsibility pipeline: read chunks, run checks, aggregate results, render output.

Module	Responsibility

readers.py	Parse JSONL, JSON, CSV chunk files into Chunk model objects
models.py	Chunk, DocumentSpec, CheckResult, IngestionAuditReport dataclasses
config.py	AuditConfig — all thresholds with sensible defaults
text_utils.py	Token n-grams, Jaccard, mojibake regex, split quality heuristics
checks.py	11 check functions, each returning a CheckResult
auditor.py	IngestionAuditor — wires checks, runs them, returns report
report.py	IngestionAuditReport — to_json(), to_markdown(), to_html()
__main__.py	CLI: python -m docingestqa chunks.jsonl --manifest ... --out ...

2.1 Status model

Each check returns one of four statuses. The overall report status is the worst status across all checks.

Status	Meaning	Action
PASS	No issue detected under configured thresholds	Safe to index
WARN	Issue detected, review before indexing	Investigate, then decide
FAIL	Serious issue, indexing should be blocked	Fix before indexing
INSUFFICIENT_INPUT	Required input missing for this check	Provide manifest

> Priority order for overall status: FAIL > WARN > INSUFFICIENT_INPUT > PASS. A single FAIL anywhere makes the report status FAIL.

3. The 11 Checks

3.1 v0.1 Checks (8)

Check	Detects	FAIL trigger
input_summary	Empty chunk set	0 chunks
metadata_completeness	Missing source or page fields	>20% chunks missing
document_coverage	Manifest docs with no chunks; orphan sources	Any missing doc
page_coverage	Pages expected by manifest but absent	Any missing page
chunk_length	Empty, short (<80 chars), long (>1500 chars) chunks	>10% empty
ocr_noise	Replacement chars, junk runs, non-printable ratio	>20% noisy
duplicate_chunks	Exact SHA-1 duplicates + near-dups via Jaccard on 5-grams	>30 pairs
source_distribution	One source > 80% of all chunks	WARN only

3.2 v0.2 Checks (3 new)

chunk_overlap

Detects consecutive chunks from the same source with high token n-gram Jaccard similarity. This catches sliding-window splitter artifacts where the step size is too small relative to the window, producing near-identical adjacent chunks.

Method: for each source, sort chunks by page, then compare each consecutive pair using Jaccard similarity on 4-grams. Pairs with Jaccard >= 0.40 are flagged as moderate overlap; >= 0.70 as high overlap.

Jaccard	Classification	Status trigger
>= 0.70	High overlap	FAIL if >= 15 pairs
>= 0.40	Moderate overlap	WARN if >= 3 total flagged
< 0.40	Normal	No flag

encoding_health

Detects four encoding failure modes:

Pattern	Cause	Example
Null bytes (\x00)	Binary artifact in text extraction	\x00\x00text\x00
BOM marker (U+FEFF)	UTF-8 file read without BOM strip	\ufeffThe document...
Mojibake	UTF-8 decoded as latin-1	cafÃ© instead of cafe
Control characters	Non-printable chars in text stream	\x0b\x0clines

The mojibake regex matches the most common UTF-8/latin-1 misread patterns: Ã[0x80-0xBF] covers 2-byte UTF-8 sequences (e.g. Ã© for e-acute), and a[0x80][0x90-0x9F] covers 3-byte smart-quote sequences. A single null byte anywhere is an immediate FAIL.

split_quality

Detects three classes of poor chunk boundaries:

Pattern	Detection	Indicates
Mid-sentence start	Chunk begins with lowercase or conjunction	Splitter cut mid-sentence
Mid-sentence end	Chunk ends without terminal punctuation	Continuation was truncated
Navigation fragment	Short chunk matching page numbers, TOC	Header/footer noise

Navigation fragments are defined as chunks under 60 characters matching patterns like bare integers, 'Page N of M', 'Table of Contents', 'Index', or 'Appendix'. These contain no semantic content and corrupt retrieval rankings.

4. Real Demo Output

From examples/audit_demo.py on a 64-chunk, 10-document synthetic corpus with deliberately seeded defects:

Check	Status	Finding
input_summary	PASS	64 chunks across 10 sources
metadata_completeness	WARN	2 chunks missing source/page metadata
document_coverage	PASS	0 missing documents, 0 orphan sources
page_coverage	FAIL	10 missing pages across 2 documents
chunk_length	WARN	3 short chunks detected
ocr_noise	WARN	2 chunks show OCR noise
duplicate_chunks	WARN	1 exact duplicate pair
source_distribution	PASS	10 sources, largest at 14.1%
chunk_overlap	PASS	1 high-overlap pair (below threshold)
encoding_health	WARN	2 chunks with mojibake (Ã© pattern)
split_quality	WARN	4 chunks with poor split boundaries

Overall: FAIL (4 PASS, 6 WARN, 1 FAIL). The FAIL is page_coverage — 10 missing pages from onboarding_guide.pdf (pp 4-5) and research_paper.pdf (p 7). This is the correct result: missing pages are a hard failure that should block indexing.

Seeded defects in the demo corpus and which checks caught them:

Seeded defect	Check that caught it	Status
Missing pages in 2 docs	page_coverage	FAIL
2 chunks with no source/page	metadata_completeness	WARN
Exact duplicate chunk pair	duplicate_chunks	WARN
Mojibake: Ã© in research paper	encoding_health	WARN
Mid-sentence start in compliance doc	split_quality	WARN
Bare page number '4'	split_quality	WARN
'Table of Contents' fragment	split_quality	WARN
OCR noise: Ch4pt3r 3	ocr_noise	WARN
Short 3-word chunks	chunk_length	WARN

5. API and Usage

5.1 Python API

The core interface is IngestionAuditor, which takes a chunks path, optional manifest path, and AuditConfig. Calling .run() returns an IngestionAuditReport.

Method	Returns
auditor.run()	IngestionAuditReport
report.to_json(path=None)	JSON string; writes file if path given
report.to_markdown(path=None)	Markdown string with check-by-check tables
report.to_html(path=None)	Self-contained HTML report
report.to_dict()	Full payload dict matching the JSON schema

5.2 CLI

python -m docingestqa chunks.jsonl --manifest source_manifest.json --out outputs/

Exits with code 0 on PASS/WARN, code 1 on FAIL. Suitable for use as a CI gate in ingestion pipelines.

5.3 Chunk format

Each line of the JSONL file is a JSON object with: chunk_id (optional), source (optional but recommended), page (optional but recommended), text (required). Chunks missing source or page are flagged by metadata_completeness but still audited by all text-based checks.

5.4 AuditConfig — key thresholds

Parameter	Default	Controls
min_chunk_chars	80	Short chunk threshold
max_chunk_chars	1500	Long chunk threshold
overlap_jaccard_warn_threshold	0.40	Moderate overlap flag
overlap_jaccard_fail_threshold	0.70	High overlap flag
warn_overlap_pair_count	3	WARN threshold for flagged pairs
fail_overlap_pair_count	15	FAIL threshold for high-overlap pairs
mojibake_warn_rate	0.05	WARN if 5% of chunks have mojibake
mojibake_fail_rate	0.20	FAIL if 20% of chunks have mojibake
warn_bad_split_rate	0.10	WARN if 10% of chunks have poor splits
fail_bad_split_rate	0.30	FAIL if 30% of chunks have poor splits

6. Methodology Notes

6.1 Why Jaccard on token n-grams, not embeddings?

Token n-gram Jaccard is deterministic, fast (O(n) per pair), has no model dependency, and is interpretable — you can explain exactly why two chunks were flagged as overlapping. Embedding cosine similarity would require a model, adds a heavy dependency, and can flag semantically similar chunks that are not ingestion artifacts. Overlap detection is about structural ingestion errors, not semantic similarity.

6.2 Why zero-dependency beyond jinja2?

DocIngestQA is a pre-indexing audit tool. It should be installable in any Python environment, including minimal Docker containers that don't have numpy or pandas. Jinja2 is required only for HTML report rendering; JSON and Markdown output are pure stdlib. The only stdlib modules used are hashlib, re, math, collections, string, unicodedata, json, and pathlib.

6.3 Why is the interpretation note mandatory?

DocIngestQA checks structural ingestion quality. It cannot tell you whether a chunk is semantically relevant to any query, whether an answer generated from it is faithful, or whether the RAG system as a whole is correct. Confusing ingestion quality with RAG correctness is a common mistake. The note is unsuppressable to prevent users from presenting DocIngestQA output as a RAG evaluation result.

6.4 What DocIngestQA does not do

Not in scope	Why
Document parsing	Audits chunks you already generated, not original files
Retrieval evaluation	No query-document relevance assessment
Answer faithfulness	No LLM-based hallucination detection
Statistical significance	Heuristic thresholds, not hypothesis tests
Embedding quality	Works on raw text only
Semantic coherence	Planned for v1.0 as optional embedding check

7. Anticipated Interview Questions

What problem does DocIngestQA solve that existing tools don't?

Generic data quality tools (Great Expectations, Pandera) work on tabular data — they don't understand chunk-specific failure modes like missing page coverage, sliding-window overlap artifacts, or mojibake corruption. RAG evaluation frameworks (RAGAS, TruLens) evaluate retrieval and generation quality but require a running system and query set. DocIngestQA is the gap between them: structural quality of the chunk corpus, before a system exists.

Why flag mid-sentence starts as a split quality issue?

A chunk beginning with 'and must be acknowledged before accessing production systems' has no subject. A generation model given this as context will either hallucinate the missing subject or produce a grammatically broken answer. The split quality check flags these so they can be investigated before indexing — either the splitter needs a larger minimum chunk size, or sentence-boundary splitting should be enforced.

What is mojibake and how do you detect it?

Mojibake is garbled text produced when a UTF-8 document is decoded using latin-1 or windows-1252. The UTF-8 bytes for e-acute (U+00E9) are 0xC3 0xA9. When each byte is decoded as latin-1, they produce the two characters Ã and copyright-symbol. The regex matches these systematic misread patterns. It cannot catch all encoding corruption, but the patterns it matches are almost never intentional in prose text, so false positive rates are very low.

Why does page_coverage use one-chunk-per-page rather than full coverage?

Requiring at least one chunk per expected page is an intentionally coarse check. A page with one chunk might still be poorly extracted, but a page with zero chunks is definitely missing. The coarse check catches hard failures (parser crashed on a page, encrypted page skipped, scanned image not OCR'd) without false-positiving on legitimately sparse pages like title pages or blank separators.

How does the CLI integrate into a CI pipeline?

python -m docingestqa chunks.jsonl exits with code 1 if the overall status is FAIL and code 0 for PASS or WARN. This makes it a drop-in gate in any CI system: add it after the chunking step, fail the pipeline on FAIL status, continue on WARN but log the report. The --out flag writes JSON/Markdown/HTML artifacts as CI pipeline evidence.

Why not just use Great Expectations or Pandera on the chunk dataframe?

Great Expectations and Pandera validate tabular data schemas and value ranges. They can check that a 'text' column is non-null or that 'page' is an integer. They cannot check whether two consecutive chunks from the same source are near-identical (sliding-window artifact), whether the text contains mojibake encoding corruption, whether a chunk starts mid-sentence, or whether page 7 of a 10-page document has zero chunks while all other pages have five. These are chunk-corpus-specific quality concepts that have no equivalent in tabular data validation. DocIngestQA is not a replacement for GE/Pandera — you could use GE for schema validation and DocIngestQA for semantic/structural ingestion quality. They operate at different layers.

Why not just use RAGAS or TruLens to evaluate the RAG system end-to-end?

RAGAS and TruLens require a running RAG system, an embedding model, and a query set to evaluate against. They measure retrieval recall and answer faithfulness on actual queries. DocIngestQA requires none of these — just the JSONL chunk export. You can run DocIngestQA immediately after chunking, before you have built any retrieval or generation layer. The two tools are complementary: DocIngestQA catches structural problems before indexing; RAGAS catches retrieval and generation problems after the system is running. If DocIngestQA finds 10 missing pages, fixing those before indexing is cheaper than debugging retrieval failures on those pages in production.

Could a missing page just mean the document was intentionally short?

Yes — this is the key limitation of page_coverage. If the manifest says a document has 10 pages but the document has a blank page 7, a correctly-functioning parser might produce zero chunks for that page. DocIngestQA would flag it as missing. This is a false positive. The check is intentionally noisy: it is better to flag a blank page and have a human confirm than to silently miss a page that failed to extract. The WARN/FAIL threshold can be tuned in AuditConfig, and the issue list includes the specific page numbers so the investigation is quick.

What would v1.0 add?

Three concrete additions. First, a semantic coherence check using embedding cosine similarity within a chunk — if the first and second half of a chunk have very low cosine similarity, it's likely two unrelated passages were concatenated by the splitter. This requires an optional embedding dependency (sentence-transformers) so it's opt-in. Second, per-source report sections: today the report aggregates all issues; a large corpus audit needs to be navigable by document. Third, configurable severity overrides so teams can promote specific checks to FAIL or demote them to WARN without forking the library. The CLI exit code design already supports severity-based gating.

8. Truth Boundary

DocIngestQA is a solo-built, open-source Python library.

It does **not** claim:

> Retrieval quality improvement
> Answer faithfulness or hallucination reduction
> Statistical significance of reported issue rates
> Complete coverage of all possible ingestion failure modes
> Compatibility with all document formats or chunking libraries

Every check in the library is a heuristic. Thresholds in AuditConfig are defaults that can and should be tuned to the specific corpus. A PASS result from DocIngestQA means no flagged issues under the configured thresholds — it does not mean the chunks are ready for production without any other review.

Resume-Safe Claim

Built DocIngestQA, a pip-installable pre-indexing QA auditor for RAG document ingestion pipelines. The library runs 11 deterministic checks — including missing page detection, OCR noise, duplicate chunks, encoding corruption (mojibake), sliding-window overlap artifacts, and split quality — on exported chunk JSONL files and produces structured JSON/Markdown/HTML reports with chunk-level evidence. Published to PyPI. Includes a CLI, a demo corpus of 64 chunks with deliberately seeded defects, and a full test suite.